



Documentação do Projeto

CaronaUni — Plataforma de Caronas Universitárias · Unoesc



Semestre 2026/1



Unoesc Chapecó



Engenharia de Software



Versão 2.0 — Evolutiva



Descrição



Escopo



Requisitos



Arquitetura



Protótipo



Modelo ER



Rastreabilidade



Plano

1. Descrição do Sistema

O **CaronaUni** é uma plataforma web de caronas universitárias desenvolvida para conectar, de forma simples e segura, estudantes da **Unoesc (Universidade do Oeste de Santa Catarina)** que oferecem vagas em seus veículos com estudantes que precisam de deslocamento até o campus.

A plataforma resolve problemas recorrentes na comunidade acadêmica: a dificuldade de encontrar caronas confiáveis, a ausência de um processo organizado para solicitar e confirmar vagas, e a falta de histórico e avaliação para aumentar a confiança entre usuários.



Propósito

- Facilitar o deslocamento diário da comunidade acadêmica
- Reduzir custos individuais de transporte
- Diminuir o número de veículos em circulação
- Promover sustentabilidade no campus



Como funciona

1. Usuário faz login via SSO Unoesc (mock)
2. Motorista cria uma carona com rota e horário
3. Passageiro busca e solicita uma vaga
4. Motorista aceita ou recusa a solicitação
5. Após a carona, ambos se avaliam

2. Escopo do Sistema

✓ Incluso (O que o sistema tem)

- Login e identificação de usuário (SSO mock)
- Cadastro de carona com rota, horário e vagas
- Mapa interativo para definição de trajeto
- Busca de caronas disponíveis com filtros
- Solicitação de vaga pelo passageiro
- Aceite ou recusa da solicitação pelo motorista
- Registro de avaliação pós-carona
- Histórico de caronas realizadas
- Gerenciamento de perfil e veículos
- Sistema de notificações (simulado)
- Caronas recorrentes por dia da semana

✗ Fora de Escopo

- Pagamento integrado
- App mobile nativo completo
- Otimização avançada de rotas (Waze/Maps)
- Integração real com SSO da Unoesc
- Chat em tempo real (WebSocket)
- Backend e banco de dados reais
- Notificações push nativas

📌 **Modalidade:** Continuação evolutiva do projeto do semestre anterior. Foco em documentação evolutiva, modelo ER e protótipo funcional navegável. Modelo de execução: **Ágil Incremental (iterativo)**.

3. Requisitos do Sistema

3.1 Requisitos Funcionais

ID	Requisito	Prioridade	Status
RF-P1	Login de usuário via SSO Unoesc (real ou mock) com identificação e autenticação	Alta	✓ Implementado
RF-P2	Criar carona com origem, destino, data/hora, número de vagas e preferências do motorista	Alta	✓ Implementado
RF-P3	Buscar caronas disponíveis e visualizar detalhes da carona e do motorista	Alta	✓ Implementado
RF-P4	Solicitar vaga em carona com mensagem opcional para o motorista	Alta	✓ Implementado
RF-P5	Aceitar ou recusar solicitação com atualização automática de vagas e notificação	Alta	✓ Implementado

ID	Requisito	Prioridade	Status
RF-P6	Avaliar usuário após carona concluída com nota, critérios detalhados e comentário	Média	✓ Implementado
RF-P7	Visualizar histórico de caronas (realizadas, futuras, pendentes de avaliação)	Média	✓ Implementado
RF-P8	Gerenciar perfil do usuário, veículos cadastrados e preferências	Média	✓ Implementado

3.2 Requisitos Não Funcionais

ID	Requisito	Categoria
RNF-P1	API e banco de dados com arquitetura simples em camadas (Frontend → API REST → BD)	Arquitetura
RNF-P2	Comunicação segura (HTTPS) e proteção de dados dos usuários	Segurança
RNF-P3	Validação para evitar inconsistência de vagas (race condition)	Consistência
RNF-P4	Interface responsiva com feedback claro de sucesso e erro para o usuário	Usabilidade

4. Arquitetura do Sistema

O sistema segue uma arquitetura em camadas com separação entre Frontend, Backend e Dados, conforme o diagrama de componentes definido na modelagem.

Frontend (Interface Web)

Protótipo em HTML5, CSS3 e JavaScript puro. Responsivo e navegável. Mapa interativo via Leaflet.js + OpenStreetMap.

HTML/CSS/JS

Leaflet.js

Backend (API REST)

Camada de serviços responsável pela lógica de negócio: autenticação, caronas, solicitações e avaliações.

Node.js / Python

REST API

Banco de Dados

SSO Institucional

Integração com o sistema de login da Unoesc para autenticação e validação de matrícula

Armazenamento relacional das entidades:
Usuário, Carona, Solicitação, Avaliação e Mensagem.

PostgreSQL / MySQL

ativa.

Mock no protótipo

Fluxo de dados: Interface Web → HTTPS → API REST → Serviços (Carona / Usuário / Mensagem)
→ Banco de Dados
Autenticação: Interface Web → SSO Unoesc → Token JWT → API REST

5. Protótipo Navegável

O protótipo foi desenvolvido em HTML/CSS/JavaScript puro, sem dependências de build, e está hospedado no GitHub Pages para acesso público.



Acessar Protótipo CaronaUni

<https://viniciusoliveira-ai.github.io/Site-Carona/>



Telas implementadas:



Login

SSO Unoesc + formulário manual



Dashboard

Estatísticas, notificações, atalhos



Buscar Carona

Mapa + lista de caronas + filtros



Oferecer Carona

Mapa interativo + formulário



Detalhes

Perfil do motorista + rota + avaliações



Solicitações

Aceitar / recusar passageiros



Avaliação

Estrelas + critérios + comentário



Histórico

Caronas passadas e futuras



Perfil

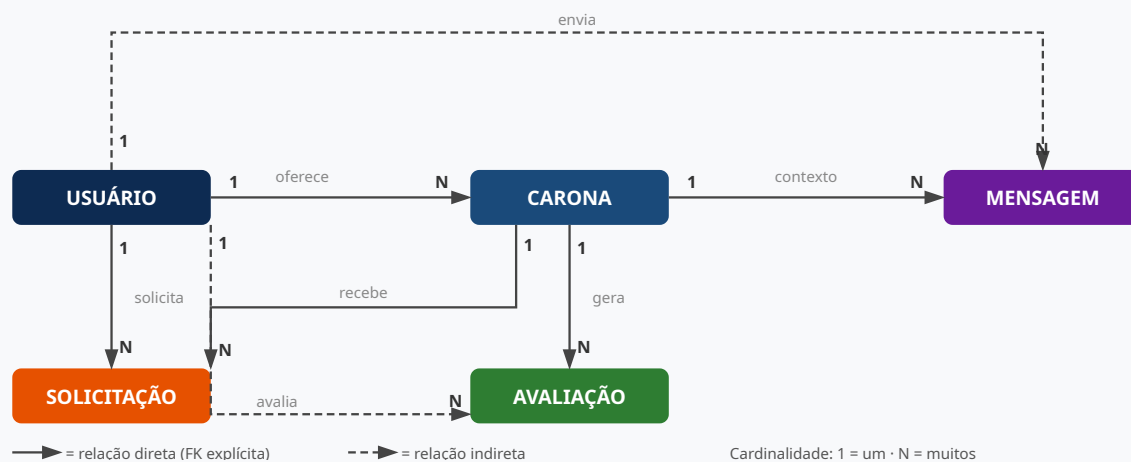
Dados, veículos, preferências

Critério de pronto atendido: Fluxo ponta a ponta funcionando (login → buscar → solicitar → aceitar → avaliar), mapa navegável com definição de origem/destino, protótipo hospedado e acessível via link público.

6. Modelo Entidade-Relacionamento (ER)

O modelo ER a seguir representa a estrutura de dados do sistema, derivada do Diagrama de Classes e dos requisitos funcionais levantados.

DIAGRAMA DE RELACIONAMENTOS



ESTRUTURA DAS ENTIDADES

USUÁRIO	
PK id	INT AUTO
nome	VARCHAR(120)
email	VARCHAR(150) UNIQUE
senha_hash	VARCHAR(255)
reputação	DECIMAL(3,2)
matricula	VARCHAR(20) UNIQUE
created_at	TIMESTAMP

CARONA	
PK id	INT AUTO
FK motorista_id	→ usuario.id
origem	VARCHAR(255)
destino	VARCHAR(255)
data_hora	DATETIME
vagas_total	TINYINT
vagas_disp	TINYINT
status	ENUM(ativa, concluída, cancelada)
created_at	TIMESTAMP

MENSAGEM	
PK id	INT AUTO
FK remetente_id	→ usuario.id
FK destinat_id	→ usuario.id
FK carona_id	→ carona.id
conteúdo	TEXT
data_hora	DATETIME

SOLICITAÇÃO

PK id	INT AUTO
FK carona_id	→ carona.id
FK passageiro_id	→ usuario.id
status	ENUM(pendente, aceito, recusado)
mensagem	TEXT NULLABLE
created_at	TIMESTAMP

AVALIAÇÃO

PK id	INT AUTO
FK avaliador_id	→ usuario.id
FK avaliado_id	→ usuario.id
FK carona_id	→ carona.id
nota	TINYINT (1 a 5)
comentario	TEXT NULLABLE
created_at	TIMESTAMP

PK = Chave Primária **FK** = Chave Estrangeira (referência a outra tabela)

AVALIAÇÃO possui **2 FKs** para USUÁRIO (avaliador e avaliado)

MENSAGEM possui **2 FKs** para USUÁRIO (remetente e destinatário)

7. Matriz de Rastreabilidade

Relação entre os requisitos funcionais e as modelagens e telas do protótipo.

Requisito	Caso de Uso	Diagrama de Sequência	Diagrama de Atividade	Tela no Protótipo
RF-P1 – Login	Fazer Login	Autenticação	—	index.html
RF-P2 – Criar carona	Criar Carona	—	Criar Carona	criar.html
RF-P3 – Buscar carona	Buscar Carona	—	Buscar e Solicitar	buscar.html / detalhes.html
RF-P4 – Solicitar vaga	Solicitar Vaga	Solicitar Carona	Buscar e Solicitar	detalhes.html
RF-P5 – Aceitar/Recusar	Aceitar Solicitação	Solicitar Carona	Processar Solicitação	solicitacoes.html
RF-P6 – Avaliar	Avaliar Usuário	Avaliar Carona	—	avaliacao.html
RF-P7 – Histórico	—	—	—	historico.html
RF-P8 – Perfil	Editar Perfil	—	—	perfil.html

8. Plano de Execução

- Iteração 1 — Levantamento e Modelagem**
Revisão dos requisitos do semestre anterior. Atualização dos diagramas de caso de uso, classes, sequência e atividades. Definição do escopo da entrega.
- Iteração 2 — Protótipo (Login + Dashboard + Buscar)**
Desenvolvimento das telas de login, dashboard principal e busca de caronas com mapa interativo Leaflet.js + OpenStreetMap.
- Iteração 3 — Protótipo (Criar + Solicitar + Aceitar)**
Tela de criação de carona com mapa, fluxo de solicitação de vaga, tela de gerenciamento de solicitações do motorista.
- Iteração 4 — Protótipo (Avaliação + Histórico + Perfil)**
Tela de avaliação pós-carona com critérios detalhados, histórico de caronas com abas e tela de perfil com preferências.
- Iteração 5 — Modelo ER e Documentação**
Elaboração do modelo ER compatível com o fluxo principal, documentação evolutiva do projeto e publicação do protótipo via GitHub Pages.
- Próximo — Backend (API REST + Banco de Dados)**
Implementação real da API com Node.js/Python, banco de dados PostgreSQL e integração com SSO da Unoesc.